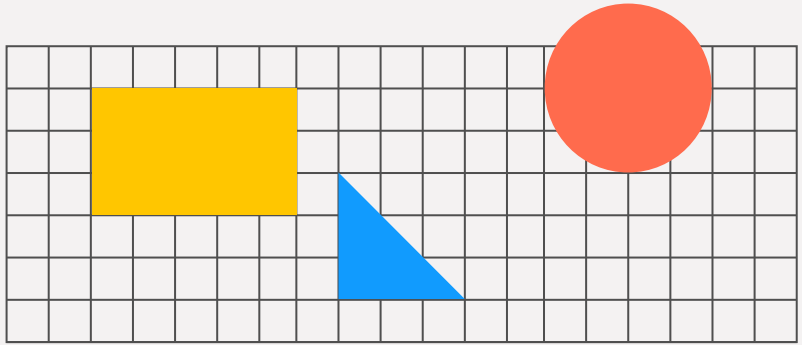
Unified Agent Architecture

Understanding the AI world with MCP and A2A protocols



About Me

Sankalp Kotewar

Nagpur

Software Engineer at Microsoft

Previously,
Github & Fidelity Investments

CSE 2018, VNIT Nagpur

Current work areas -

Sovereign Cloud, DevOps, AI

Hobbies

Travelling
Photography
Music
Designing
Gaming
Browsing the Internet!

Agenda

Level 0 - Chat

Level 1 - RAG

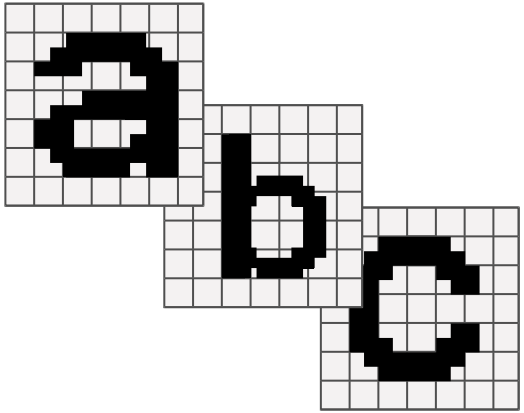
Level 2 - MCP

Level 3 - A2A & Beyond...

Learning Outcome

Goals

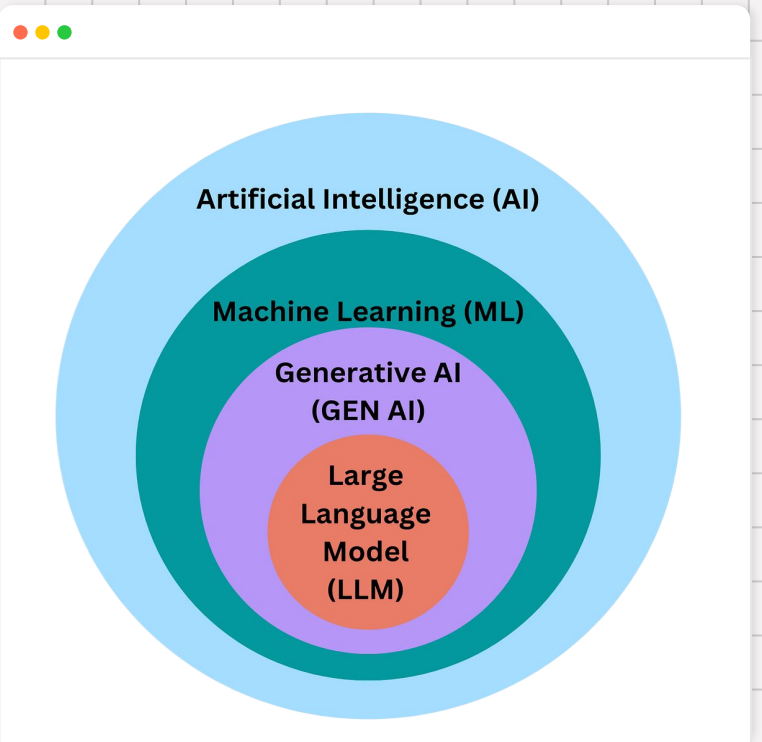
- Understand LLMs
- How RAG came into the picture?
- Why MCP & A2A?
- Questions?

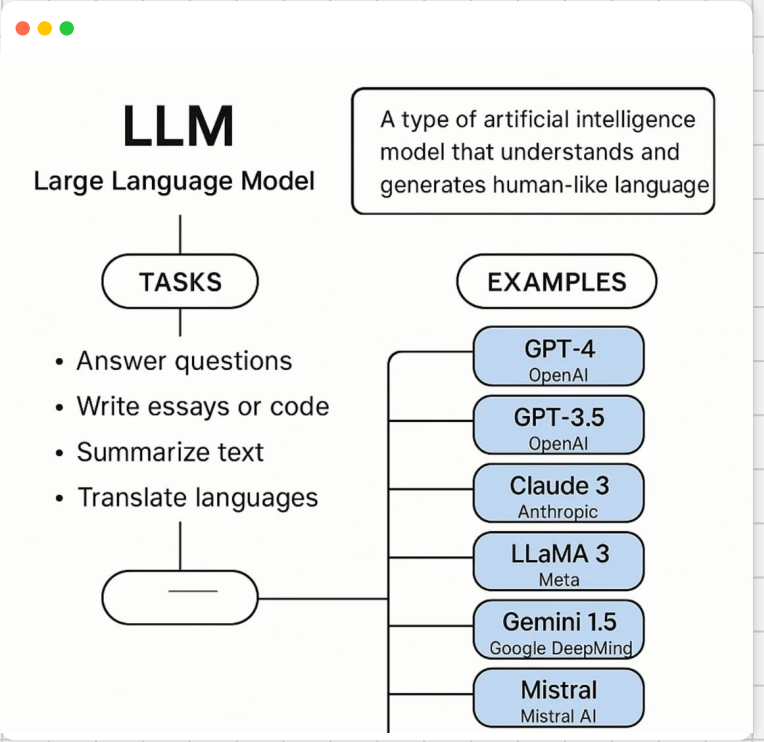


Level 0

What is an LLM?

Large Language Models





What LLM is & isn't

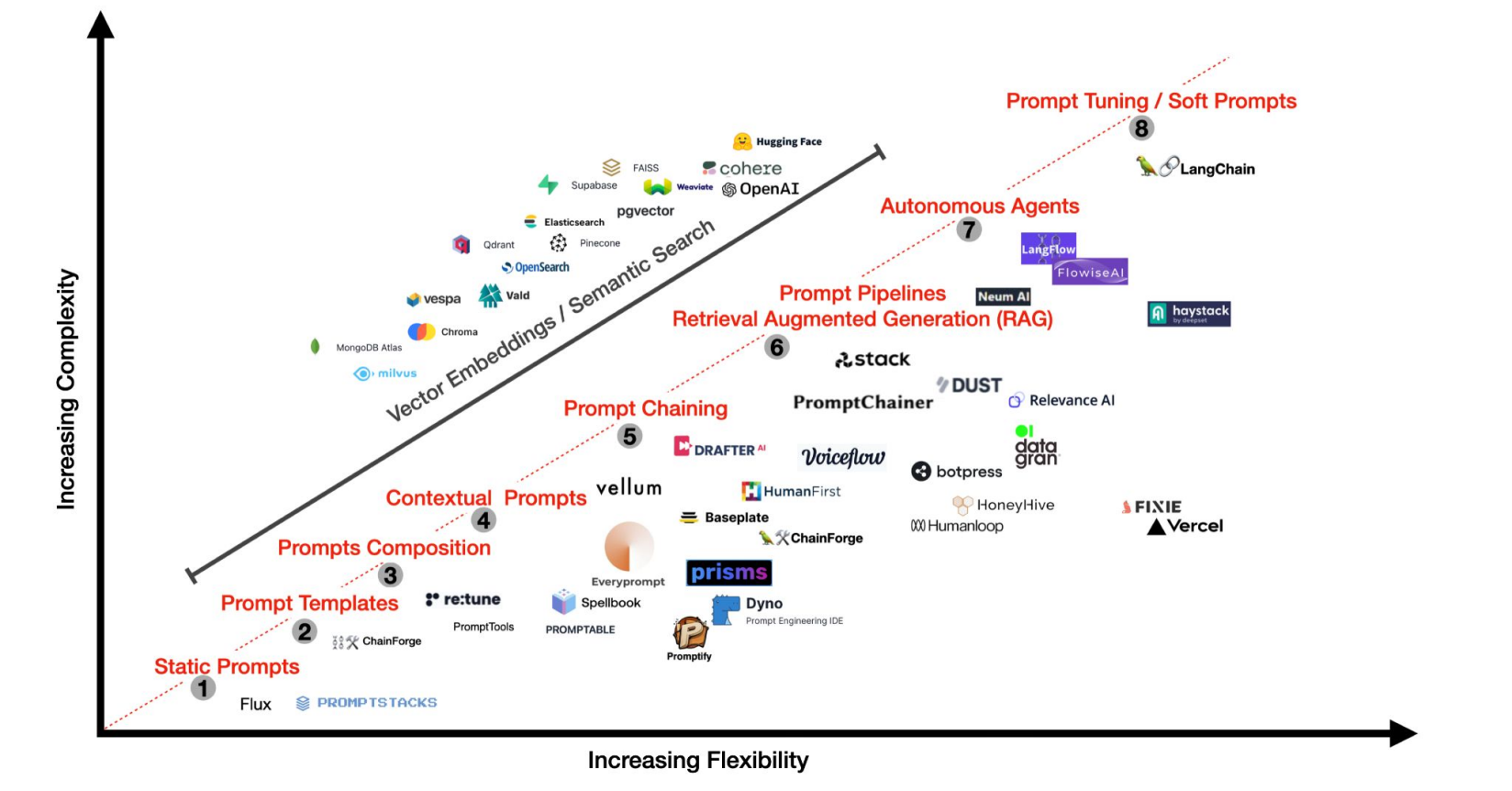
What it is

- A statistical prediction engine that calculates the most likely next word.
- A compressed representation of patterns found in its training data.
- Fluent and plausible sounding text (which may or may not be true).
- A mathematical optimization to satisfy the user's prompt.

What is isn't

- A source of truth - it can "hallucinate" with total confidence.
- A logical calculator that inherently understands math or physics rules.
- A database or a search engine that "looks up" stored files.
- LLM isn't everything in AI - its a subset!

How LLMs evolved with time



Level 1

RAG, but why?

What is RAG?

Retrieval — Find relevant information

Augmented — Add it to AI's knowledge

Generation — Create a better response

How RAG changes things?

RAG transforms LLMs from **"closed-brain"** systems into **"open-book"** systems.

By providing specific source documents, RAG significantly reduces "**hallucinations**"

Without RAG:

- The AI might give a generic "standard" industry answer

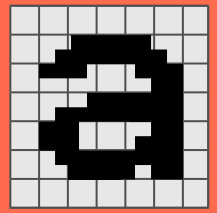
With RAG:

- **Research:** The system searches the internal Wiki and recent Slack threads.
- **Audit:** It filters out an old policy in favor of the new updates.
- **Generate:** It provides the exact steps: "To deploy to the xyz region, use the v2-secure-gateway as per the latest March update."

Use RAG whenever you want your LLM to look for specific data that's private and something the model isn't trained on.

Tips

Until now...

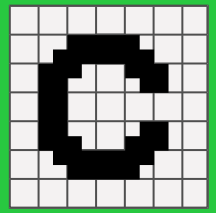


LLM

Models can answer to your questions in the chat based on the context...

RAG

You can feed context specific to your use-case to get more accurate answers



But?

There are still many limitations...

Limitation

Biggest hurdle

LLMs have a cutoff date, they're not trained on data beyond their cutoff date.

So how do we get the data realtime?

An Intermediate Solution
Functions & tooling

Function (Function Calling): The *instruction*. It is the LLM's ability to output structured data (usually a JSON payload) to trigger a specific action. It's the model realizing it needs help and saying, "Run this specific piece of code with these exact parameters."

Tooling (Tools): The *equipment*. These are the actual external programs, APIs, or utilities (like a web search browser, a Python interpreter, or a database) connected to the LLM. They execute the function and feed the real-world results back to the model.

Supporting your own functions and toolings is a good way to enhance LLMs but there was no standard & lot of duplication...

The problem

Level 2

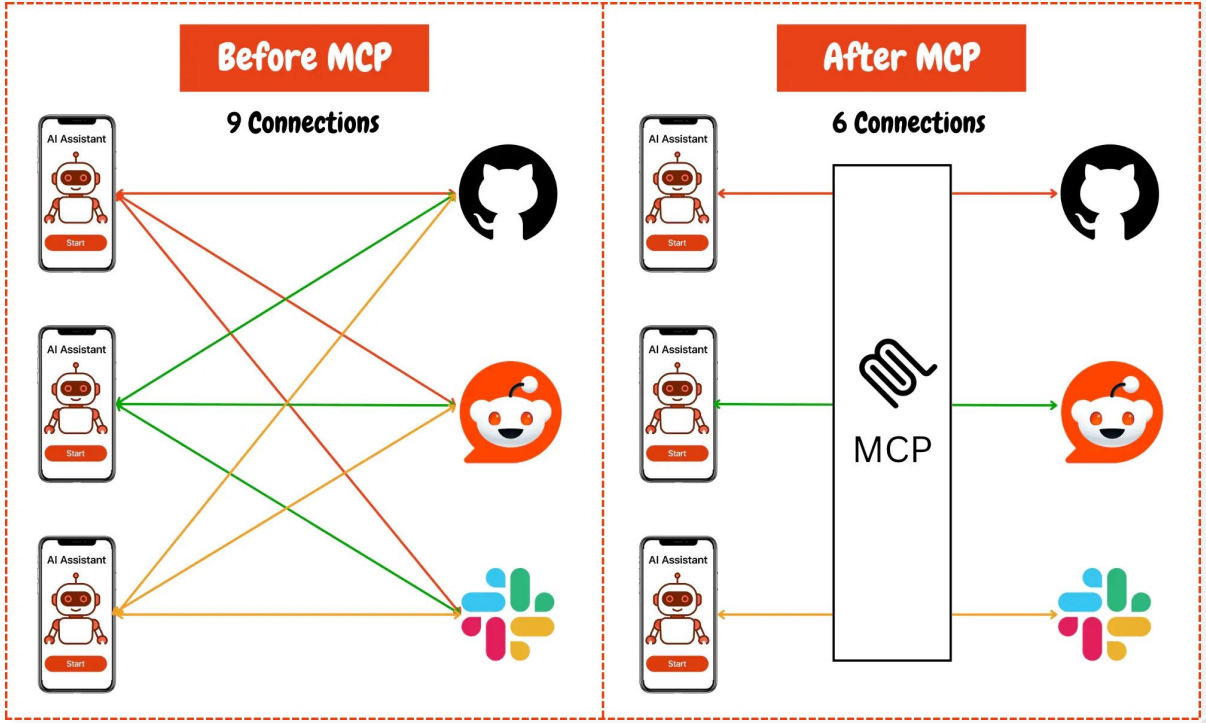
Let’s explore MCP

The problem

N X M problem

N agents × M tools = N·M custom integrations.

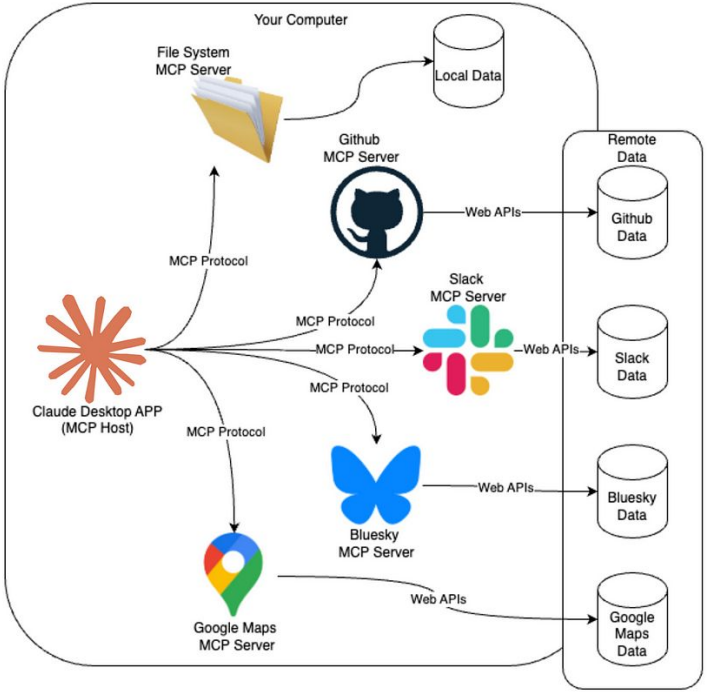
Protocols reduce this to N + M adapters



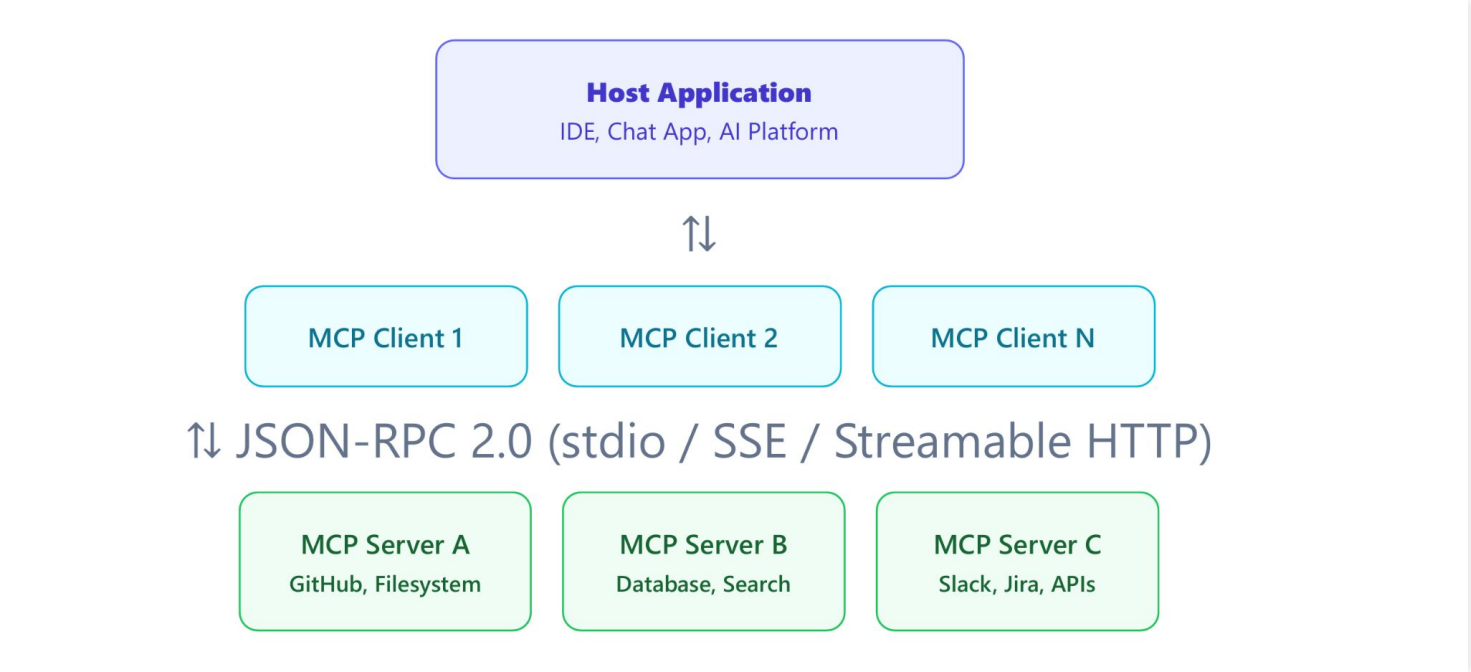
The Solution

MCP Protocol

An open standard (by Anthropic) that provides a universal interface between AI models and external capabilities.



The MCP Architecture



MCP Lifecycle

Initialize — Client sends capabilities, server responds with its capabilities

Initialized — Client confirms; ready for requests

Operation — Bidirectional requests, responses & notifications flow

Shutdown — Clean teardown of transport layer

A small MCP Demo

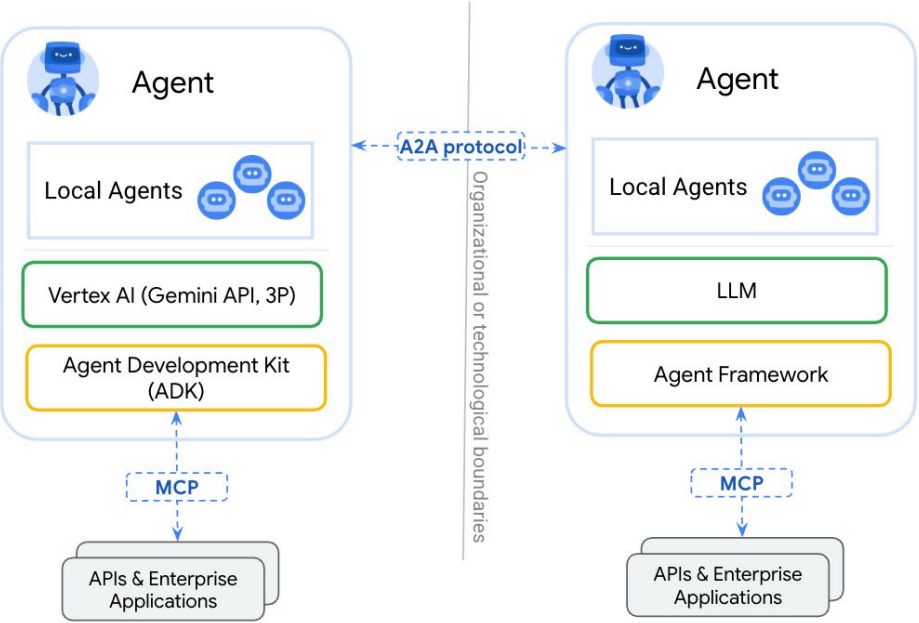
We'll integrate a to-do list hosted in a SQLite DB

Level 3

Let’s explore A2A

The A2A Protocol

An open standard for seamless AI agent communication and collaboration.

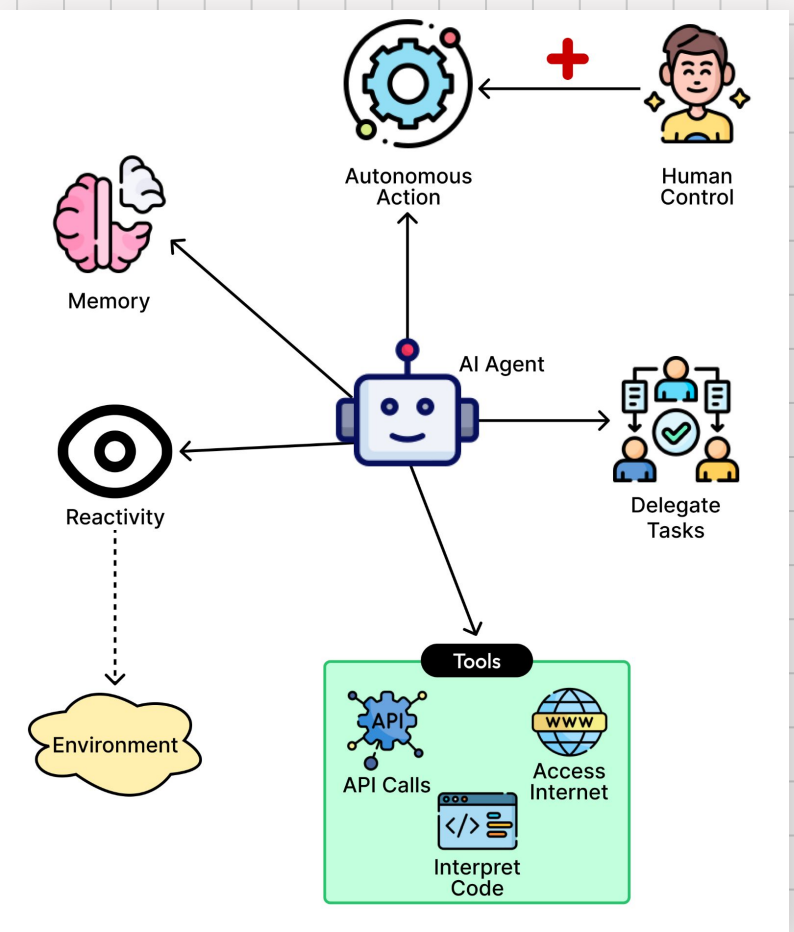


What is AI agent?

An **AI agent** is an autonomous system that uses a **Large Language Model** (LLM) as its "brain" to perceive its environment, reason through complex tasks, and take actions to achieve a specific goal.

The Core Components

- **Perception:** Receiving input (text, images, or sensor data).
- **Reasoning:** Planning the sequence of steps needed to reach the goal.
- **Action:** Using external tools or APIs to interact with the world.
- **Memory:** Storing past interactions to refine future decisions.



Chatbot v/s AI Agent

Feature	Normal LLM (Chatbot)	AI Agent
Operational Mode	**Reactive:** Responds to a single prompt and stops.	**Proactive:** Breaks a goal into tasks and loops until finished.
Capability	**Knowledge-based:** Predicts the next token based on training.	**Action-oriented:** Can browse the web, edit files, or call APIs.
Reasoning	**Linear:** Follows a straight path from input to output.	**Iterative:** Can self-correct, "think" before acting, and evaluate results.
Memory	**Context Window:** Limited to the current conversation.	**Long-term Memory:** Often uses databases (RAG) to remember facts across weeks.

Reasoning v/s Agency

Common misconception

Reasoning and Agency are two different "muscles" in the AI world.

Reasoning v/s Agency

Think of it this way:

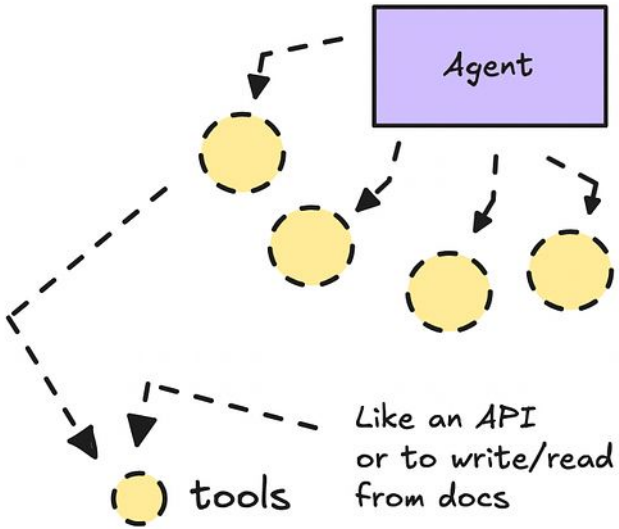
Reasoning is the ability to think through a complex logic puzzle (like a high-IQ student taking a silent exam).

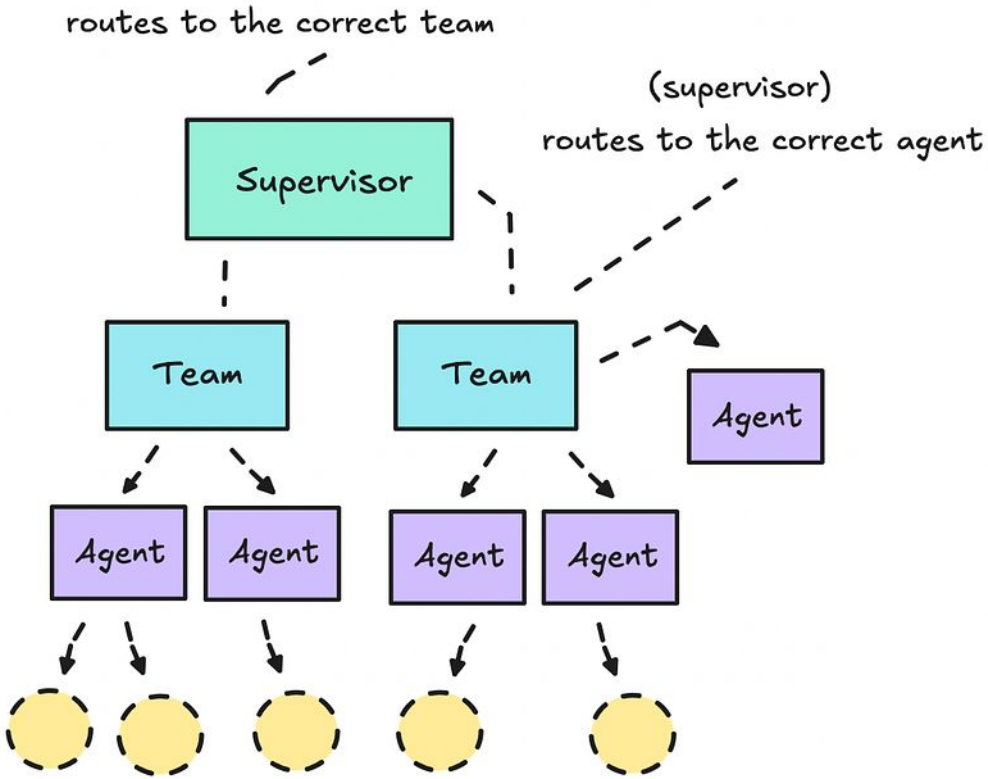
Agency is the ability to act in the real world to solve a goal (like an intern actually running an errand).

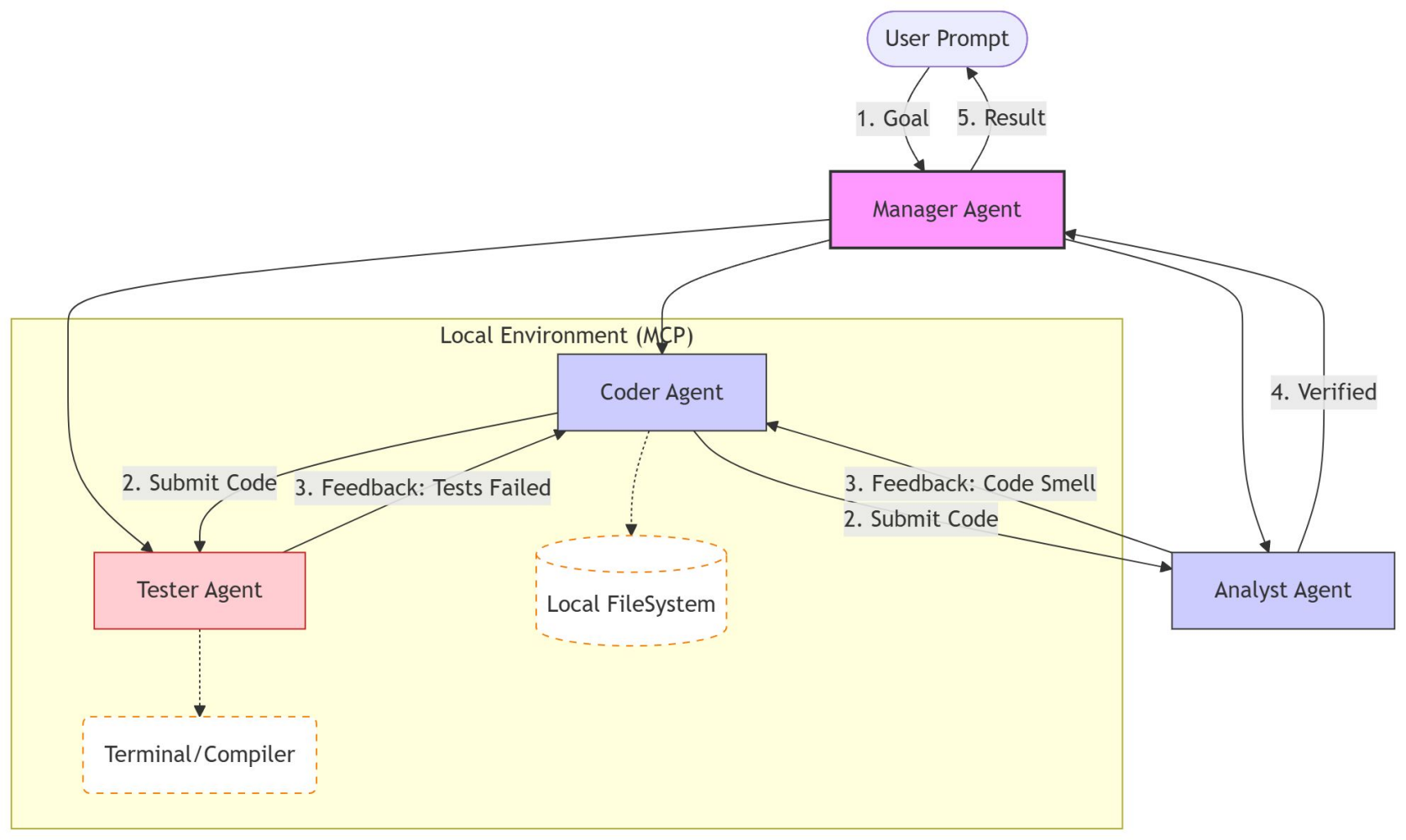
The evolution of LLMs

Timeline Phase	Key Capability & Icon	Technical Milestone
Pre-Agent Era (Pre-2023)	**Static Knowledge🧠**	**Standard LLM (GPT-3):** Predicts next tokens based purely on training data. Text-in, text-out.
The Ingestion Era (Late 2023)	**Grounded Memory (RAG) 📚**	**LLM + Vector DB:** Model retrieves your private data (PDFs, docs) before answering to reduce hallucinations.
The Action Era (2024)	**Tool Execution (MCP)🛠️**	**Function Calling / Model Context Protocol:** Model outputs structured JSON to trigger APIs, edit local files, or query live databases.
The Reasoning Era (Early 2025)	**Autonomous Planning🎯**	**Reasoning Models (o1/R1):** Internal "Chain of Thought" allows the agent to create multi-step plans and self-correct mistakes before taking action.
The Collaborative Era (2026+)	**Agent-to-Agent (A2A)🤝**	**Unified Agentic AI:** A "Manager Agent" uses MCP to act, and standard A2A protocols to discover and delegate tasks to specialized "Worker Agents."

Single vs Multiagent Systems






User Prompt
1. Goal
5. Result
Manager Agent
Local Environment (MCP)
Coder Agent
4. Verified
2. Submit Code
3. Feedback: Tests Failed
3. Feedback: Code Smell
2. Submit Code
Tester Agent
Local FileSystem
Analyst Agent
Terminal/Compiler

The Problem

Interoperability

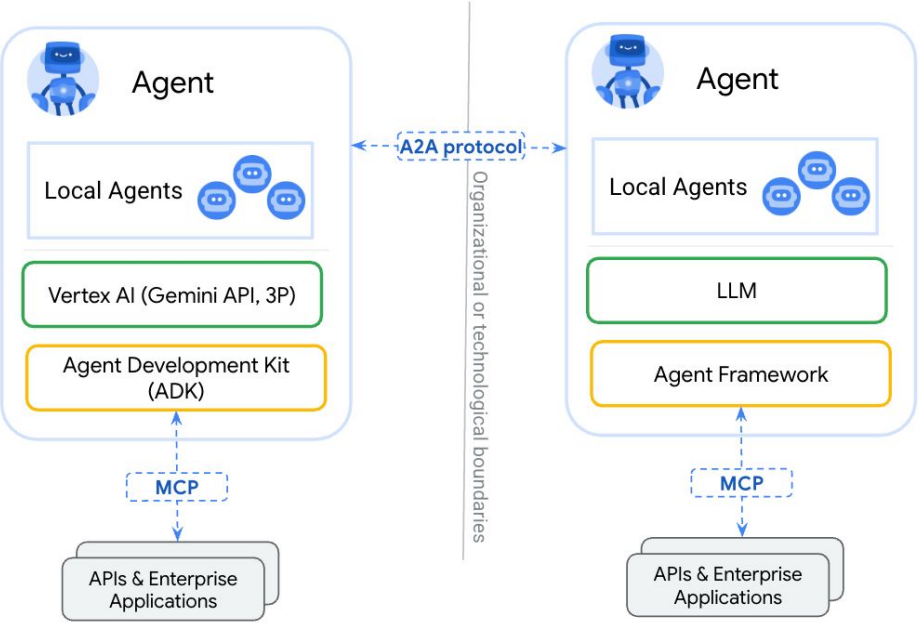
A universal agentic layer where any agent can discover, negotiate with, and delegate to any other agent or tool

The A2A Protocol

A2A Protocol
An open standard for seamless AI agent communication and collaboration.

Interoperability
Provides a common language to bridge diverse frameworks and vendor silos.

Unified Ecosystem
Enables agents from different developers and organizations to work together.



Key capabilities

Capability Discovery: Uses machine-readable "Agent Cards" (JSON) to advertise services and help clients select the best agent for a task.

Task Management: Tracks task lifecycles and statuses to ensure agents stay synchronized during long-running operations.

Collaboration: Facilitates back-and-forth messaging and artifact sharing to maintain context and conversation flow.

UX Negotiation: Adapts content delivery by negotiating supported formats (like video or web forms) based on the client’s UI capabilities.

In other words

Feature	Core Question
Capability Discovery	"What can you do?"
Task Management	"What is the status?"
Collaboration	"What is the context?"
UX Negotiation	"How should I display this?"

Core components

Agent Card: A JSON "résumé" (at a well-known URL) detailing an agent's skills, endpoints, and auth requirements.

A2A Server: The remote agent that exposes HTTP endpoints, handles tasks, and advertises its skills via Agent Cards.

A2A Client: The orchestrator that identifies suitable servers and distributes tasks based on their specific capabilities.

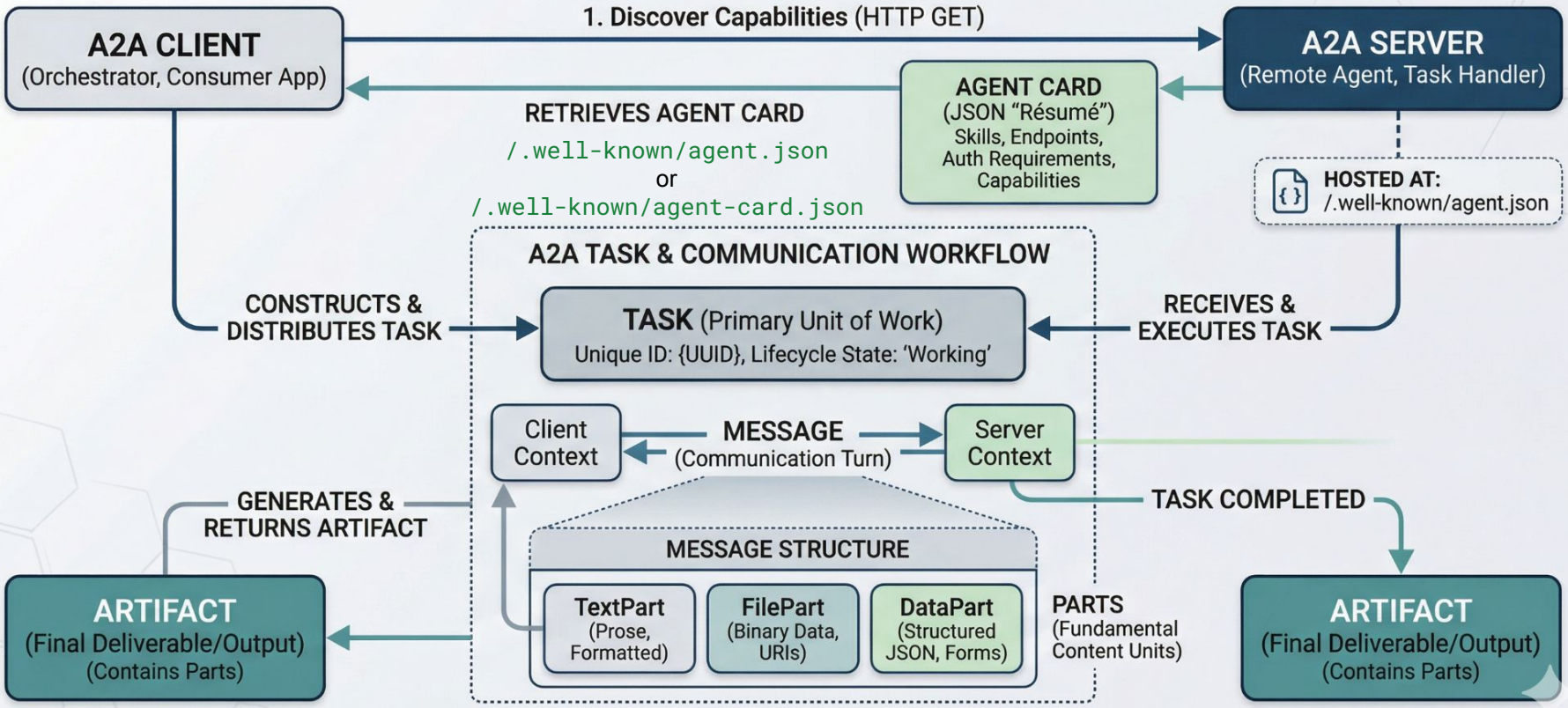
Task: The primary unit of work with a unique ID and a defined lifecycle (e.g., submitted, working, completed).

Message: Individual communication turns within a task used to exchange data between client and server.

Part: The basic content unit within messages, categorized as **TextPart** (prose), **FilePart** (binary/URI), or **DataPart** (JSON).

Artifact: The final output or deliverable generated by the server and returned to the client.

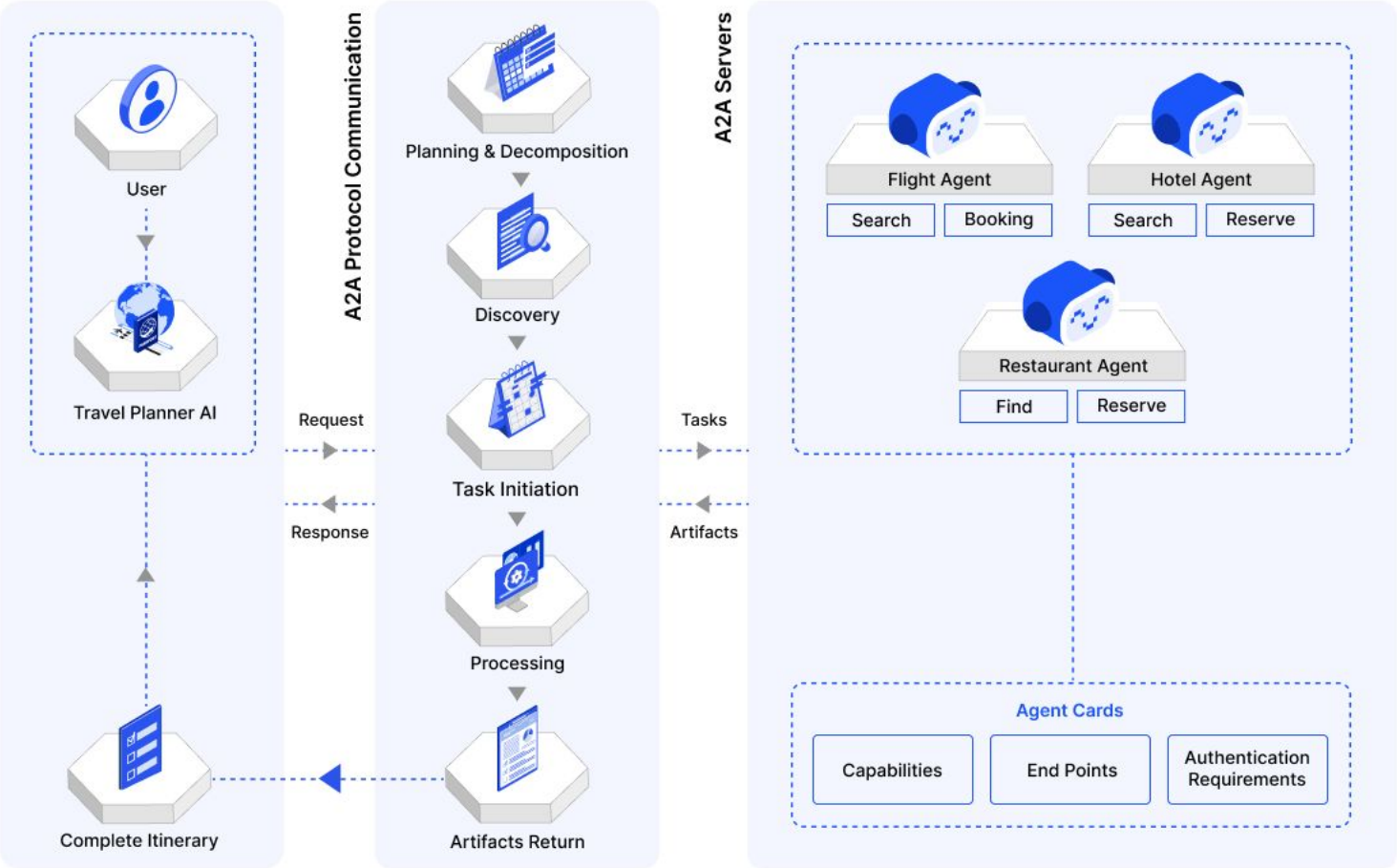
A2A (Agent-to-Agent) Architecture: Component Interactions



Sample agent card

```
{
  "name": "Task Manager Agent",
  "description": "An LLM-powered A2A agent that manages tasks with SQLite persistence. Understands natural language — no rigid commands needed. Other agents
can delegate task management work to this agent.",
  "url": "http://127.0.0.1:9999",
  "version": "1.0.0",
  "defaultInputModes": ["text"],
  "defaultOutputModes": ["text"],
  "capabilities": {
    "streaming": true
  },
  "skills": [
    {
      "id": "task_management",
      "name": "Task Management",
      "description": "Manages a persistent task list. Can add, list, complete, delete, and summarize tasks. Understands natural language requests.",
      "tags": ["tasks", "productivity", "todo", "project-management"],
      "examples": [
        "Please add a high-priority task to fix the login bug",
        "What tasks are currently pending?",
        "Mark task 1 as done",
        "Remove task 3 from the list"
      ]
    }
  ]
}
```

A2A Protocol Workflow



Level 4

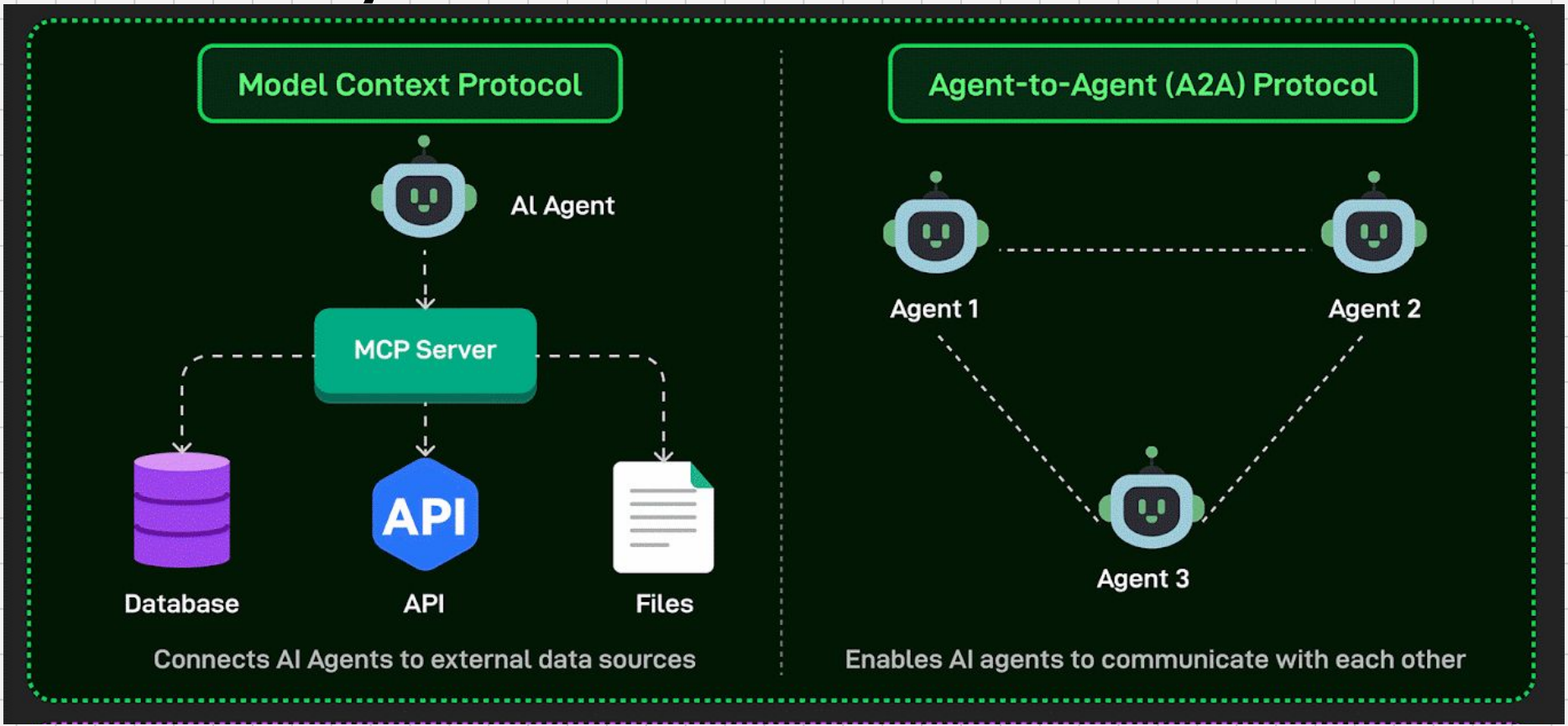
MCP or A2A?

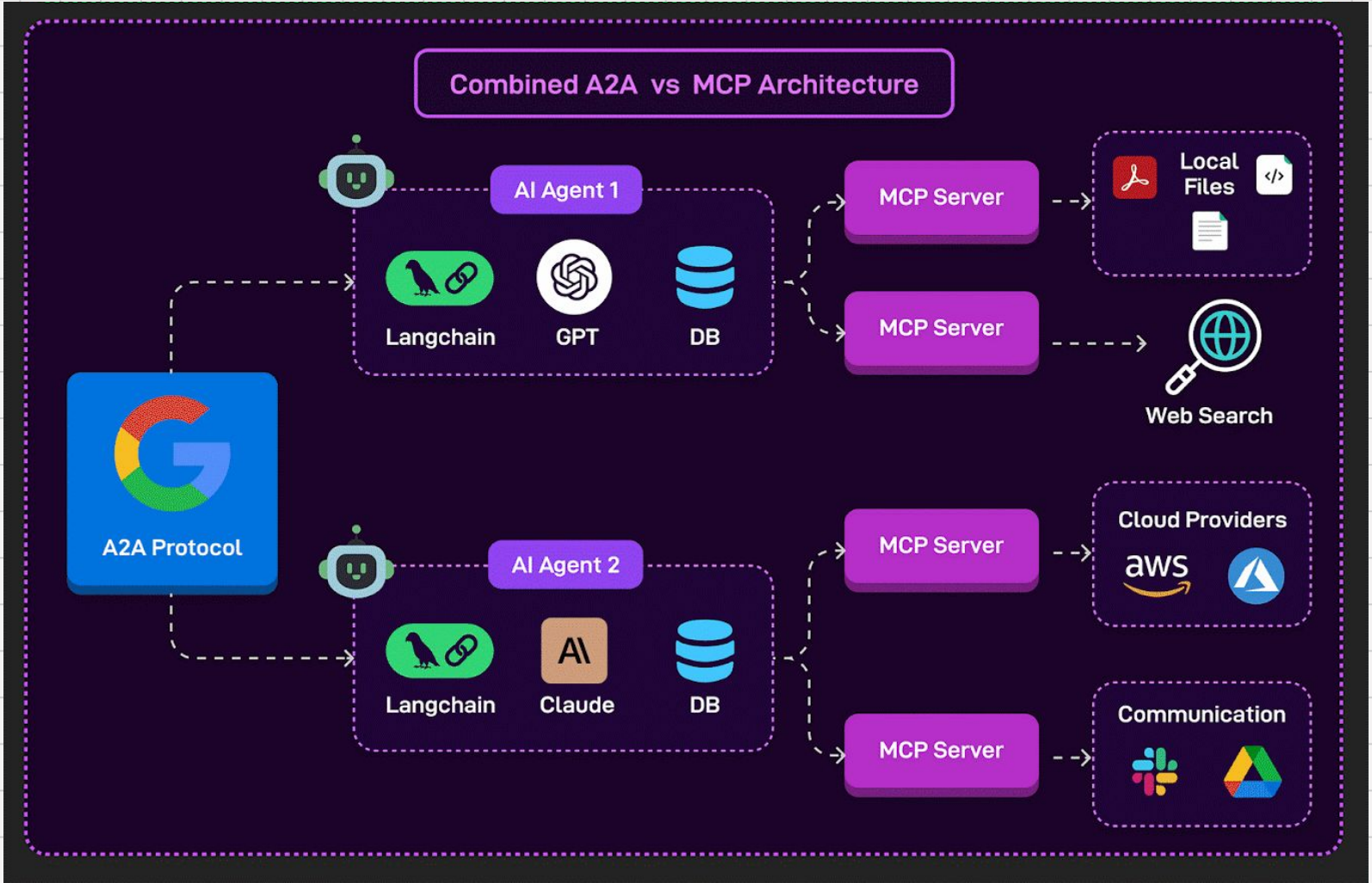
MCP or A2A - which is better?

They aren't competitors!

They are two different layers of the "Unified Agentic" stack.

How they differ?




Combined A2A vs MCP Architecture
AI Agent 1
Langchain
GPT
DB
MCP Server
MCP Server
Local Files
Web Search
A2A Protocol
AI Agent 2
Langchain
Claude
DB
MCP Server
MCP Server
Cloud Providers
aws
Communication

Questions?

Thank you!

Slides download link

Connect with me

@sankalpkotewar

Slides download link

Connect with me

@sankalpkotewar